

Digital Nurture 5.0

Python Full Stack Engineer Track

HANDS-ON EXERCISE BOOK

Frontend Development: HTML5, CSS3, JavaScript, React, Angular & Vue.js
10 Hands-On Exercises

How to Use This Exercise Book

This exercise book covers Module 2 — Frontend Development Basics for the Digital Nurture 5.0 Deep Skilling Program. The 10 hands-on exercises are organised to progressively build your frontend skills — starting from HTML and CSS fundamentals, moving through JavaScript, then into React, Angular, and Vue.js frameworks, and finally covering accessibility, cross-browser compatibility, and state management.

All exercises share a single running project — a Student Portal web application — so your work builds on itself naturally from one exercise to the next.

Software & Tools Required

- VS Code — <https://code.visualstudio.com/download>
- Node.js (LTS) + npm — <https://nodejs.org/en/download/>
- Git — <https://git-scm.com/downloads>
- A modern browser (Chrome or Firefox recommended) with DevTools
- Browser-based editors for quick practice: CodePen (codepen.io), CodeSandbox (codesandbox.io), StackBlitz (stackblitz.com)
- Vue SFC Playground — <https://play.vuejs.org/>
- Angular on StackBlitz — <https://stackblitz.com/fork/angular>

Submission Guidelines

- Organise solutions in a folder named `Module2_FrontendDev/<YourName>/`
- Each hands-on gets its own subfolder: `handson_01/`, `handson_02/`, ...
- For framework exercises (React, Angular, Vue): include the full project folder minus `node_modules`
- Push to your GitHub repo and share the URL with your POC

NOTE	Hands-On 1 through 4 are standalone HTML/CSS/JS files — no framework needed. Hands-On 5 onwards require Node.js and npm.
-------------	--

Difficulty Guide

Level	Hands-On	What to Expect
Beginner	1, 2, 3	HTML5 structure, CSS3 layouts, Responsive design
Intermediate	4, 5, 6	Async JS, Fetch API, React components & hooks
Advanced	7, 8, 9, 10	Angular, Vue.js, Accessibility, State management

Common Scenario: Student Portal Web Application

All exercises build one unified project — a Student Portal. Each hands-on adds a new layer to this portal, so by the end you will have a working, multi-page, accessible, responsive frontend application.

The Student Portal is a web interface where students can view their profile, browse enrolled courses, check grades, and receive notifications. You will build the UI layer progressively — first with plain HTML/CSS/JS, then migrate features into React, Angular, and Vue.js to understand how frameworks solve real problems.

Pages / Sections in the Portal

Page / Section	Built In	Key Elements
Home / Landing	Hands-On 1 & 2	Header, nav bar, hero section, footer
Course Listing	Hands-On 2 & 3	Card grid layout, responsive columns
Student Profile	Hands-On 3 & 4	Form, DOM interaction, async data load
Notifications	Hands-On 4	Live API fetch, dynamic DOM rendering
React SPA	Hands-On 5 & 6	Components, hooks, routing, state
Angular App	Hands-On 7	Services, DI, reactive forms, routing
Vue.js App	Hands-On 8	Composition API, Vue Router, Pinia
Accessible Portal	Hands-On 9	ARIA, keyboard nav, WCAG checks
State Management	Hands-On 10	Redux / NgRx / Pinia patterns

HANDS-ON 1 [Beginner]**HTML5 Semantic Structure & CSS3 Foundations****Topics Covered:**

✓ HTML5 Semantic Elements	✓ CSS3 Selectors & Specificity
✓ CSS Box Model	✓ Typography & Spacing
✓ Basic Page Layout	

In this first hands-on, you will build the structural skeleton of the Student Portal using proper semantic HTML5 and style it with CSS3 fundamentals. No frameworks — just clean, well-structured HTML and CSS.

TOOL	Create a single folder <code>handson_01/</code> with two files: <code>index.html</code> and <code>styles.css</code> . Open <code>index.html</code> directly in your browser.
-------------	--

Task 1: Build the Page Skeleton with Semantic HTML5

Goal: Create a well-structured, accessible HTML page using the correct semantic elements.

Steps:

1. Create `index.html` with the proper HTML5 doctype, `<html lang='en'>`, `<head>` with `charset UTF-8`, `viewport` meta tag, and a descriptive `<title>`.
2. Inside `<body>`, create the following semantic sections: `<header>` (site name + navigation), `<main>` with two child sections: `<section id='hero'>` and `<section id='courses'>`, and `<footer>`.
3. Inside `<header>`, add a `<nav>` with an unordered list of links: Home, Courses, Profile, Grades.
4. Inside the hero section, add an `<h1>` with 'Welcome to the Student Portal', a short `<p>` description, and a `<button>Explore Courses</button>`.
5. Inside the courses section, add an `<h2>`, and three `<article>` elements each containing an `<h3>` (course name), `<p>` (description), and a `<span>` showing credits.
6. Validate your HTML at <https://validator.w3.org/> — resolve all errors before proceeding.

Hint:

- Use `<article>` for self-contained content like course cards — it is more appropriate than `<div>` here.
- The `<main>` element should appear only once per page and wraps the primary content.

Expected Outcome: W3C validator reports zero errors. The page has a visible header, nav, two sections, and a footer when opened in a browser.

Task 2: Apply CSS3 Fundamentals

Goal: Style the page using selectors, the box model, and basic typography.

Steps:

7. Link `styles.css` to `index.html` using a `<link>` tag inside `<head>`.
8. Apply a CSS reset: set `margin: 0`, `padding: 0`, `box-sizing: border-box` on the universal selector (`*`).
9. Style the `<body>`: set font-family to a system font stack (e.g., Arial, sans-serif), a background colour, and a default text colour.
10. Style the `<header>`: add padding, a distinct background colour, and use `display: flex` with `justify-content: space-between` to separate the site name and nav links.
11. Style the nav links: remove `list-style`, display them inline, add spacing between items, and change the text-decoration.
12. Style the hero section: centre the text with `text-align: center`, add generous padding, and style the button with padding, a border, and a hover state using the `:hover` pseudo-class.
13. Add a CSS class `.course-card` to each article and apply padding, a border, border-radius, and a box-shadow.

Hint:

- box-sizing: border-box makes width calculations intuitive — padding and border are included inside the declared width.
- Use browser DevTools (F12) to inspect the box model of any element in real time.

Expected Outcome: The page is visually styled with a clear header, styled navigation, hero with a button, and bordered course cards.

HANDS-ON 2 [Beginner]**CSS Flexbox, Grid & Responsive Design****Topics Covered:**

✓ CSS Flexbox	✓ CSS Grid
✓ Mobile-First Design	✓ Media Queries
✓ Viewport Units & Fluid Layouts	

You will now take the Student Portal from Hands-On 1 and make it fully responsive using Flexbox for the navigation and header, CSS Grid for the course cards, and media queries to adapt the layout across screen sizes.

| Task 1: Flexbox Navigation & Header Layout

Goal: Use Flexbox to create a professional, aligned header and navigation bar.

Steps:

14. Revisit the `<header>` in your existing `index.html` and `styles.css` from Hands-On 1.
15. Apply `display: flex` on the header. Use `align-items: center` and `justify-content: space-between`.
16. Style the `<nav>` as a flex container with `gap` between items. Ensure links are vertically centred.
17. Create a hero section using Flexbox: stack the heading, paragraph, and button vertically with `align-items: center` and a column flex-direction.
18. Add a student stats bar below the hero with three stat items (e.g., Courses Enrolled: 3, GPA: 3.8, Semester: 6). Lay them out using Flexbox with equal spacing.

Hint:

- `flex: 1` on a child makes it grow to fill available space.
- `gap` property works in both Flexbox and Grid and is cleaner than `margin` for spacing flex children.

Expected Outcome: The header, nav, and stats bar all align correctly side-by-side on a wide viewport.

| Task 2: CSS Grid Course Card Layout

Goal: Use CSS Grid to build a responsive multi-column course card layout.

Steps:

19. Wrap all three course `<article>` elements in a `<div class='course-grid'>`.
20. Apply `display: grid` on `.course-grid` with `grid-template-columns: repeat(3, 1fr)` and a `gap`.
21. Give each `.course-card` a minimum height and make it stretch to fill its grid cell using `align-self: stretch`.
22. Add two more course cards (total 5) and observe how the grid automatically places them.
23. Use `grid-template-columns: repeat(auto-fit, minmax(280px, 1fr))` — observe how the grid adapts as you resize the browser window.

Hint:

- `auto-fit` with `minmax` creates a self-responsive grid that requires no media queries for column reflow.
- CSS Grid is ideal for 2D layouts (rows and columns); Flexbox is better for 1D (row or column only).

Expected Outcome: The 5 course cards display in a 3-column grid on wide viewports and reflow automatically as the window narrows.

| Task 3: Responsive Design with Media Queries

Goal: Apply mobile-first design principles and media queries to support all screen sizes.

Steps:

24. Rewrite your existing CSS to be mobile-first: start with a single-column layout as the default (no media queries), then use min-width media queries to enhance larger screens.

25. Add a media query at min-width: 768px: change the course grid to 2 columns and show the full nav bar (replace a hamburger placeholder text with the actual nav links).
26. Add a media query at min-width: 1024px: change the course grid to 3 columns and increase hero section padding.
27. Use viewport units: set the hero section min-height: 40vh and the site title font-size to clamp(1.5rem, 3vw, 2.5rem) for fluid typography.
28. Open DevTools and use the device toolbar to test the layout at 375px (mobile), 768px (tablet), and 1280px (desktop). Fix any overflow or layout breaks.

Hint:

- Mobile-first means: write base styles for the smallest screen, then add complexity with min-width breakpoints — not max-width.
- clamp(min, preferred, max) lets font sizes scale fluidly between breakpoints without abrupt jumps.

Expected Outcome: The portal looks correct at 375px (single column, stacked nav), 768px (2-column cards), and 1280px (3-column cards, full nav).

HANDS-ON 3 [Beginner]**JavaScript ES6+ & DOM Manipulation****Topics Covered:**

✓ let / const / var	✓ Arrow Functions & Template Literals
✓ Array Methods (map, filter, reduce)	✓ DOM Selection & Modification
✓ Event Listeners	✓ ES6 Modules (import/export)

You will now bring the Student Portal to life with JavaScript. This hands-on focuses on modern ES6+ syntax and direct DOM manipulation — no frameworks yet.

TOOL	Create handson_03/. You can continue building on your HTML/CSS from earlier, or start fresh with a new index.html that links to app.js using <code><script type='module' src='app.js'></script></code> .
-------------	--

Task 1: ES6+ Syntax Practice

Goal: Rewrite common JavaScript patterns using modern ES6+ features.

Steps:

29. Create a data.js file and export a const array of 5 course objects, each with id, name, code, credits, and grade properties.
30. In app.js, import the array using ES6 import. Use destructuring to extract name and credits from each course in a loop.
31. Use Array.map() to create a new array of strings formatted as 'CS101 — Data Structures (4 credits)'. Log the result.
32. Use Array.filter() to get only courses with credits >= 4. Log the count.
33. Use Array.reduce() to calculate the total credits enrolled. Log the result.
34. Rewrite an existing for loop in your code as an arrow function. Use a template literal for string interpolation.

Hint:

- const does not mean immutable for objects/arrays — it means the binding cannot be reassigned. You can still push to a const array.
- Destructuring: `const { name, credits } = course;` — much cleaner than `course.name`, `course.credits` separately.

Expected Outcome: Console shows the formatted course strings array, the filtered list, and the total credits (e.g., 18).

Task 2: DOM Selection & Dynamic Rendering

Goal: Select elements, modify the DOM, and render data dynamically.

Steps:

35. In index.html, ensure the course grid `<div class='course-grid'>` exists but is empty (remove hardcoded course articles).
36. In app.js, use `document.querySelector('.course-grid')` to select the grid container.
37. Loop through your course data array. For each course, create a `<article>` element using `document.createElement()`, set its `className`, and build its inner HTML using a template literal with course name, code, and credits.
38. Append each created article to the course grid using `appendChild()`.
39. Add a `<p id='total-credits'>` below the grid. After rendering, update its `textContent` to display the total credits dynamically.

Hint:

- innerHTML is fine for simple content you control. For user-generated content, use `textContent` to prevent XSS.
- `DocumentFragment` can batch-append multiple nodes in one DOM update — better performance than repeated `appendChild` calls.

Expected Outcome: The course grid is fully rendered from JavaScript data. The total credits paragraph shows the correct sum.

| Task 3: Event Listeners & Interactivity

Goal: Add user interactions using event listeners and DOM updates.

Steps:

40. Add a text `<input id='search-courses' placeholder='Search courses... '>` above the course grid.
41. Add an event listener on the input's 'input' event. On each keystroke, filter the course array by name (case-insensitive) and re-render only the matching cards.
42. Add a 'Sort by Credits' `<button>`. On click, sort the courses array by credits descending and re-render the grid.
43. Add a click event to each course card: when clicked, display an alert (or update a `<div id='selected-course'>`) showing the course name and grade.
44. Use event delegation: attach a single click listener to the course-grid container and detect which card was clicked using `event.target.closest('.course-card')`.

Hint:

- Event delegation is more efficient than attaching individual listeners to each card, especially when cards are dynamically added or removed.
- Always remove or re-render carefully — clear the container with `innerHTML = ''` before re-rendering to avoid duplicate cards.

Expected Outcome: Typing in the search box instantly filters cards. Clicking Sort by Credits reorders them. Clicking a card shows its details.

HANDS-ON 4 [Intermediate]**Async JavaScript, Fetch API & API Integration****Topics Covered:**

✓ Promises & async/await	✓ Fetch API
✓ Error Handling (try/catch)	✓ Loading States
✓ Axios (introduction)	✓ Dynamic DOM from API Data

You will now replace hardcoded data with live data fetched from public APIs. This hands-on covers Promises, async/await, error handling, and loading state management — core skills for any real frontend application.

API	Use the free JSONPlaceholder API (https://jsonplaceholder.typicode.com) to simulate backend data. It provides /users, /posts, /todos — no authentication needed.
------------	---

Task 1: Promises and async/await

Goal: Understand Promise chaining and rewrite it using async/await for cleaner code.

Steps:

45. Write a function `fetchUser(id)` that calls `fetch('https://jsonplaceholder.typicode.com/users/' + id)` and returns a Promise. Chain `.then()` to parse the JSON and log the user's name.
46. Rewrite the same function using `async/await` and `try/catch`.
47. Write a function `fetchAllCourses()` that simulates a 1-second network delay using `new Promise(resolve => setTimeout(resolve, 1000))` and then returns your local course array.
48. Call `fetchAllCourses()` and render the course cards only after the promise resolves. Show a 'Loading courses...' message while waiting.
49. Demonstrate `Promise.all()`: fetch user ID 1 and user ID 2 simultaneously and log both names when both requests complete.

Hint:

- `async/await` is syntactic sugar over Promises — it does not change how the event loop works.
- `Promise.all()` is faster than sequential awaits when the requests are independent of each other.

Expected Outcome: The loading message appears for ~1 second before cards render. `Promise.all` logs both user names together.

Task 2: Fetch API with Error Handling

Goal: Build a robust data-fetching layer with proper error handling and UI feedback.

Steps:

50. Create a reusable async function `apiFetch(url)` that fetches data, checks `response.ok`, throws a descriptive Error if not ok, and returns the parsed JSON.
51. Use `apiFetch` to load posts from `/posts` and render them as notification cards in a new `<section id='notifications'>` on the page.
52. Add a loading spinner (a simple animated CSS div or a text indicator) that appears while the fetch is in-progress and disappears when data loads.
53. Simulate a 404 error by calling `apiFetch` with a bad URL (e.g., `/nonexistent`). Display a user-friendly error message in the UI — never just log to console.
54. Add a Retry button that appears when an error occurs. On click, it re-calls the fetch function and re-renders the section.

Hint:

- Always check `response.ok` — `fetch` only rejects on network errors (offline), not on HTTP errors like 404 or 500.

- A simple loading indicator: add/remove a CSS class that shows a 'Loading...' paragraph before and after the fetch.

Expected Outcome: Notifications load from the API. A bad URL shows a friendly error message and a Retry button. Retry re-fetches successfully.

| Task 3: Introduction to Axios

Goal: Compare Fetch with Axios and use Axios for API calls.

Steps:

55. Add Axios via CDN in your HTML: `<script src='https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js'></script>`.
56. Rewrite the `apiFetch` function using `axios.get(url)`. Note that Axios: (a) automatically parses JSON, (b) throws on non-2xx responses by default — no need to check `response.ok` manually.
57. Use `axios.get` with a `params` object: `axios.get('/posts', { params: { userId: 1 } })` to fetch posts belonging to user 1.
58. Add a request interceptor using `axios.interceptors.request.use()` that logs 'API call started: `<url>`' before every request.
59. Compare: write a side-by-side comment in your code listing three differences between `fetch` and `axios`.

Hint:

- Axios has a built-in request timeout option — useful in production: `axios.get(url, { timeout: 5000 })`.
- Fetch is built into browsers; Axios is a library but offers more convenience features and better defaults.

Expected Outcome: Axios fetches user 1's posts correctly. The interceptor logs every request. Your comment lists 3 clear fetch vs axios differences.

HANDS-ON 5 [Intermediate]**React Fundamentals — Components, Props, State & Hooks****Topics Covered:**

✓ JSX Syntax	✓ Functional Components
✓ Props & Prop Types	✓ useState Hook
✓ useEffect Hook	✓ Conditional Rendering & Lists

You will now rebuild the Student Portal as a React application. This hands-on focuses on the core building blocks of React: JSX, components, props, state, and the most important hooks.

SETUP	Run: <code>npm create vite@latest student-portal-react -- --template react</code> then <code>cd student-portal-react && npm install && npm run dev</code>
--------------	---

Task 1: Project Setup & First Components

Goal: Scaffold the React app and create the first reusable components.

Steps:

- Set up the Vite + React project using the command in the setup note above. Verify it runs at `http://localhost:5173`.
- Clean up the boilerplate: remove default `App.css` content, clear `App.jsx` to a minimal shell.
- Create a `components/` folder. Inside it, create `Header.jsx` — a functional component that renders the site title and a nav bar with links (Home, Courses, Profile).
- Create a `Footer.jsx` component with a copyright line.
- Import and render `Header` and `Footer` inside `App.jsx`. Pass the site name as a prop to `Header`: `<Header siteName='Student Portal' />`. Display it inside the component using `{props.siteName}`.
- Create a `CourseCard.jsx` component that accepts `name`, `code`, `credits`, and `grade` as props and renders a styled card.

Hint:

- JSX is transpiled to `React.createElement()` calls — always return a single root element or use a Fragment `<>...</>`.
- Props are read-only inside a component. Never mutate props directly.

Expected Outcome: The app renders the header with the passed site name, a footer, and at least one `CourseCard` with hardcoded prop values.

Task 2: State with useState and Dynamic Lists

Goal: Use `useState` to manage component state and render dynamic lists.

Steps:

- In `App.jsx`, create a state variable using `useState`: `const [courses, setCourses] = useState([])` and initialise it with your 5 course objects (import from a data file).
- Map over the `courses` state to render a `CourseCard` for each course: `{courses.map(course => <CourseCard key={course.id} {...course} />)}`.
- Add a search input above the course list. Create a state variable `searchTerm`. On each keystroke (`onChange`), update `searchTerm` and filter the displayed courses.
- Add an 'Enroll' button inside `CourseCard`. When clicked, add the course to a new `enrolledCourses` state array in `App.jsx` (lift state up). Pass the handler as a prop: `onEnroll={handleEnroll}`.
- Display the count of enrolled courses in the `Header` using another prop.

Hint:

- Each list item must have a unique `key` prop — use a stable ID, never the array index (index keys cause subtle bugs during reorders).

- Lifting state up means moving state to the closest common ancestor of components that need it.

Expected Outcome: Typing in search filters the course cards in real time. Clicking Enroll updates the enrolled count in the header.

| Task 3: useEffect Hook & Lifecycle

Goal: Use useEffect to fetch data and manage side effects.

Steps:

71. In App.jsx, replace the hardcoded initial state with a useEffect that fetches courses from JSONPlaceholder /posts (map the first 5 posts to course-like objects).
72. Add a loading state: `const [loading, setLoading] = useState(true)`. Set it to false after the fetch completes. Render a 'Loading...' message while loading is true.
73. Add an error state. Catch fetch errors and display an error message if the request fails.
74. Create a separate StudentProfile.jsx component with its own local state (name, email, semester). Add a form with inputs bound to state via onChange handlers.
75. Use a useEffect with a dependency array to log 'Courses updated' to the console whenever the courses state changes. Explain in a code comment why the dependency array matters.

Hint:

- An empty dependency array `[]` means the effect runs once after mount — equivalent to `componentDidMount`.
- A missing dependency array means the effect runs after every render — this can cause infinite loops if the effect itself triggers a state update.

Expected Outcome: Courses load from the API on mount. The loading message appears briefly. The profile form updates local state on typing.

HANDS-ON 6 [Intermediate]**React Routing & State Management****Topics Covered:**

✓ React Router v6	✓ useNavigate & useParams
✓ useContext Hook	✓ Context API for Global State
✓ Introduction to Redux Toolkit	

You will extend the React Student Portal with multi-page navigation using React Router, share state across components using Context API, and get an introduction to Redux Toolkit for more complex state management.

INSTALL	<code>npm install react-router-dom and npm install @reduxjs/toolkit react-redux</code>
----------------	--

Task 1: Multi-Page Navigation with React Router

Goal: Add routing to turn the app into a proper single-page application.

Steps:

76. Wrap your `<App />` in `<BrowserRouter>` in `main.jsx`.
77. Define routes in `App.jsx` using `<Routes>` and `<Route>`: `/ → HomePage`, `/courses → CoursesPage`, `/profile → ProfilePage`, `/courses/:courseId → CourseDetailPage`.
78. Update the Header nav links to use React Router's `<Link>` component instead of `<a>` tags (prevents full page reload).
79. In `CourseDetailPage.jsx`, use the `useParams()` hook to read the `courseId` from the URL and display the matching course details.
80. Add a `useNavigate()` hook: after clicking Enroll on a course, navigate the user to `/profile` automatically.

Hint:

- `<Link>` renders as an `<a>` in the DOM but intercepts the click to do a client-side navigation — no server round-trip.
- `useParams` returns an object of URL parameters — always check that the param exists before using it.

Expected Outcome: Clicking a course card navigates to `/courses/1` and shows that course's detail. Enrolling navigates to `/profile`.

Task 2: Global State with Context API

Goal: Share enrolled courses state across components without prop drilling.

Steps:

81. Create `EnrollmentContext.jsx`: define a context with `createContext()`, a provider component that holds enrolledCourses state, and exports both.
82. Wrap the app in `<EnrollmentProvider>` in `main.jsx`.
83. In any component that needs enrolled courses (e.g., Header for count, ProfilePage for list), consume the context using `useContext(EnrollmentContext)` instead of receiving props.
84. Add a Remove function to the context: allow students to un-enroll from a course from the ProfilePage.
85. Verify that enrolling on the CoursesPage and viewing the count in the Header both reflect the same state — without any props being passed between them.

Hint:

- Context solves prop drilling (passing props through many intermediate components that don't use them). It is not a replacement for all state — local state is still appropriate for UI-only concerns.
- Avoid putting rapidly-changing state in context — every context update re-renders all consumers.

Expected Outcome: Enrolling a course from CoursesPage immediately updates the count in Header and the list in ProfilePage — all via context, no props.

| Task 3: Introduction to Redux Toolkit

Goal: Refactor the enrollment state to use Redux Toolkit — the modern, recommended Redux approach.

Steps:

86. Create store.js using configureStore from @reduxjs/toolkit.
87. Create enrollmentSlice.js using createSlice with initial state { enrolledCourses: [] } and two reducers: enroll(state, action) and unenroll(state, action).
88. Replace the Context-based enrollment state with Redux: dispatch the enroll action on Enroll click, dispatch unenroll on Remove.
89. Read state in components using useSelector(state => state.enrollment.enrolledCourses).
90. Open Redux DevTools in the browser. Enroll and un-enroll a course and observe the action log and state diff.

Hint:

- Redux Toolkit (RTK) is the modern Redux — it eliminates boilerplate with createSlice and Immer for immutable updates.
- Redux DevTools extension for Chrome/Firefox is essential for debugging — install it before testing.

Expected Outcome: Redux DevTools shows enroll and unenroll actions with correct payloads. State updates correctly in all components using useSelector.

HANDS-ON 7 [Advanced]**Angular — Components, Services, DI, Routing & Forms****Topics Covered:**

✓ Angular CLI	✓ Components & Modules
✓ Services & Dependency Injection	✓ RxJS & HttpClient
✓ Angular Router	✓ Reactive Forms

You will build a standalone Angular version of the Student Portal. Angular is an opinionated, full-featured framework — this hands-on covers its core architecture: components, services, dependency injection, and routing.

SETUP	<code>npm install -g @angular/cli then ng new student-portal-angular --routing --style=css then cd student-portal-angular && ng serve</code>
--------------	--

Task 1: Angular Components & Data Binding

Goal: Create components and practise Angular's data binding techniques.

Steps:

1. Generate components using Angular CLI: `ng generate component header`, `ng generate component course-list`, `ng generate component course-card`, `ng generate component student-profile`.
2. In `CourseCardComponent`, declare `@Input()` properties: `name`, `code`, `credits`, `grade`. Render them in the template using interpolation `{{ }}`.
3. In `CourseListComponent`, define a `courses` array and render course cards using `*ngFor`: `<app-course-card *ngFor='let course of courses' [name]='course.name' ...>`.
4. Add a search input in `CourseListComponent` and use two-way binding with `[(ngModel)]` to bind it to a `searchTerm` property. Import `FormsModule` in the module.
5. Use `*ngIf` to show a 'No courses found' message when the filtered list is empty.

Hint:

- `[property]='value'` is one-way binding from component to template. `(event)='handler()'` is event binding. `[(ngModel)]` is two-way binding — it needs `FormsModule`.
- `ngFor` tracks items by index by default — use `trackBy` for better performance with large lists.

Expected Outcome: The course list renders from the array. Typing in search filters cards. Empty search shows the 'No courses found' message.

Task 2: Services, Dependency Injection & HttpClient

Goal: Create a service to centralise data-fetching logic and inject it into components.

Steps:

96. Generate a service: `ng generate service course`. In `CourseService`, inject `HttpClient` in the constructor and add a method `getCourses()` that returns an `Observable` by calling `this.http.get<any[]>('https://jsonplaceholder.typicode.com/posts?_limit=5')`.
97. Import `HttpClientModule` in `AppModule` (or `provideHttpClient()` in `bootstrapApplication` for standalone).
98. Inject `CourseService` into `CourseListComponent` via the constructor: `constructor(private courseService: CourseService) {}`.
99. In `ngOnInit()`, subscribe to `courseService.getCourses()` and assign the result to the `courses` array.
100. Add a loading boolean: set it to true before the call and false in the subscription callback. Use `*ngIf='loading'` to show a spinner.

Hint:

- Services in Angular are singletons by default (providedIn: 'root') — one instance shared across the whole app.

- Always unsubscribe from Observables in ngOnDestroy to prevent memory leaks — or use the async pipe which handles this automatically.

Expected Outcome: Courses load from the API via the service. The spinner appears briefly on page load. The component has no direct HTTP code.

| Task 3: Angular Routing & Reactive Forms

Goal: Set up routing between views and build a form using Angular Reactive Forms.

Steps:

101. In app-routing.module.ts, define routes: { path: "", component: CourseListComponent } and { path: 'profile', component: StudentProfileComponent }.
102. Add <router-outlet> in app.component.html and update header links to use [routerLink]='["/profile"]'.
103. In StudentProfileComponent, import ReactiveFormsModule and build a FormGroup with FormControl fields: name (required), email (required + email validator), semester (required, min: 1, max: 8).
104. Bind the form to the template using [formGroup]='profileForm' and formControlName on each input.
105. Display validation error messages: show 'Name is required' when name is touched and invalid. Show 'Enter a valid email' for the email field.
106. Add a Submit button disabled until the form is valid: [disabled]='profileForm.invalid'. On submit, log the form value.

Hint:

- Reactive Forms are defined in the component class (not the template) — this makes validation logic testable and explicit.
- FormGroup.value returns an object with all control values. FormGroup.valid is true only when all validators pass.

Expected Outcome: Navigating to /profile shows the form. Submitting with invalid data shows inline error messages. Submit is disabled until all fields are valid.

HANDS-ON 8 [Advanced]**Vue.js — Composition API, Vue Router & Pinia****Topics Covered:**

✓ Vue 3 Template Syntax	✓ Reactivity (ref, reactive)
✓ Components & Props	✓ Composition API (setup)
✓ Vue Router	✓ Pinia State Management

You will now build the Student Portal using Vue.js 3. Vue's Composition API, combined with Vue Router and Pinia, offers a modern, flexible approach to building SPAs.

SETUP	<code>npm create vue@latest student-portal-vue (select: Vue Router yes, Pinia yes) then cd student-portal-vue && npm install && npm run dev</code>
--------------	--

Task 1: Vue 3 Components & Reactive Data

Goal: Create Vue components and practise the Composition API with reactive state.

Steps:

107. Clean up the boilerplate. In `src/components/`, create `CourseCard.vue` and `Header.vue` as single-file components (SFC) with `<template>`, `<script setup>`, and `<style scoped>`.
108. In `CourseCard.vue`, define props using `defineProps({ name: String, code: String, credits: Number, grade: String })`. Render them in the template.
109. In a parent view (`CoursesView.vue`), create a reactive `courses` array using `ref([])`. Initialise it with 5 course objects inside `onMounted`.
110. Render `CourseCard` for each course using `v-for='course in courses' :key='course.id'`. Pass props using `v-bind shorthand :name='course.name'`.
111. Add a computed property `filteredCourses` using `computed(() => courses.value.filter(...))` that filters by a `ref` `searchTerm`. Bind an input to `searchTerm` using `v-model`.

Hint:

- In `<script setup>`, `ref()` creates a reactive reference. Access its value with `.value` in script, but directly in the template — Vue unwraps refs automatically in templates.
- `computed()` is cached — it only re-runs when its reactive dependencies change, unlike a method which runs on every render.

Expected Outcome: Courses render from reactive data. Typing in the search box filters cards via the computed property in real time.

Task 2: Vue Router for Navigation

Goal: Configure client-side routing between views in the Vue app.

Steps:

112. In `src/router/index.js`, define routes: `/` → `HomeView`, `/courses` → `CoursesView`, `/courses/:id` → `CourseDetailView`, `/profile` → `ProfileView`.
113. In `App.vue`, add `<RouterLink to='/courses'>Courses</RouterLink>` in the nav and `<RouterView />` as the page outlet.
114. In `CourseDetailView.vue`, use `useRoute()` from `vue-router` to access the `:id` param and display the matching course details.
115. Add a programmatic navigation: after clicking `Enroll` in `CourseDetailView`, use `useRouter().push('/profile')` to redirect to the profile page.
116. Add a navigation guard using `router.beforeEach`: log 'Navigating to: `<path>`' before each route change.

Hint:

- `<RouterLink>` is Vue's equivalent of React Router's `<Link>` — it prevents full page reloads.
- `useRoute()` gives you the current route object (params, query, path). `useRouter()` gives you the router instance for programmatic navigation.

Expected Outcome: All four routes work. Navigating to `/courses/2` shows course 2's detail. Enrolling redirects to `/profile`.

| Task 3: Pinia for State Management

Goal: Manage shared enrollment state using Pinia — Vue's official state management library.

Steps:

117. Create `src/stores/enrollment.js`. Define a Pinia store using `defineStore('enrollment', () => { ... })` with: state `ref enrolledCourses = ref([])`, a computed `totalCredits`, an action `enroll(course)`, and an action `unenroll(courseId)`.
118. In `CoursesView.vue`, import and use the enrollment store: `const store = useEnrollmentStore()`. Call `store.enroll(course)` on Enroll button click.
119. In `ProfileView.vue`, display `store.enrolledCourses` using `v-for` and show `store.totalCredits` as a summary.
120. In `Header.vue`, display the enrolled course count using `store.enrolledCourses.length`.
121. Open Vue DevTools in the browser. Enroll a course and inspect the Pinia state tab to see the state update.

Hint:

- Pinia stores are reactive by default — any component reading store properties will re-render automatically when they change.
- Pinia's Composition API style (`setup store`) is more flexible and TypeScript-friendly than the Options API style.

Expected Outcome: Enrolling from `CoursesView` updates the count in `Header` and the list in `ProfileView`. Vue DevTools Pinia tab shows the action and state change.

HANDS-ON 9 [Advanced]**Web Accessibility (a11y) & Cross-Browser Compatibility****Topics Covered:**

✓ WCAG 2.1 Guidelines	✓ ARIA Attributes
✓ Semantic HTML for Accessibility	✓ Keyboard Navigation
✓ Colour Contrast	✓ Feature Detection & Polyfills
✓ Cross-Browser Testing Tools	

Accessibility and cross-browser compatibility are not optional — they are professional requirements. In this hands-on, you will audit the Student Portal against WCAG 2.1 guidelines, fix identified issues, and verify compatibility across browsers.

TOOL	Use Lighthouse (built into Chrome DevTools) for automated accessibility auditing. Use axe DevTools browser extension for detailed violation reports.
-------------	--

Task 1: Accessibility Audit & Semantic Fixes

Goal: Run an accessibility audit and fix semantic HTML issues.

Steps:

122. Open the Student Portal (HTML/CSS version from Hands-On 1–3) in Chrome. Open DevTools → Lighthouse → check Accessibility → Generate report.
123. Record the initial accessibility score and list all flagged issues in a comment at the top of your index.html file.
124. Fix missing alt attributes: add descriptive alt text to all elements. Use alt="" for decorative images.
125. Fix any missing form labels: every <input> must have an associated <label> (either wrapping or via htmlFor/for attribute). Add labels to the search input and profile form fields.
126. Fix heading hierarchy: ensure headings go h1 → h2 → h3 without skipping levels. Correct any violations found by Lighthouse.
127. Replace any <div> or used as interactive elements (buttons, links) with the correct semantic element, or add role and keyboard event handling.

Hint:

- Screen readers announce the page structure using headings — a correct heading hierarchy helps blind users navigate the page efficiently.
- An with no alt attribute is an accessibility violation. alt="" (empty string) tells screen readers to skip the image — use it for purely decorative images.

Expected Outcome: Lighthouse accessibility score improves from the baseline. All images have alt text. All form inputs have associated labels.

Task 2: ARIA & Keyboard Navigation

Goal: Add ARIA attributes and ensure full keyboard navigability.

Steps:

128. Add aria-label to the <nav> element: aria-label='Main navigation'. Add aria-current='page' to the currently active nav link.
129. Make the course cards keyboard-accessible: add tabindex='0' and a keydown event listener so pressing Enter on a focused card triggers the same action as a click.
130. Add role='status' and aria-live='polite' to the search results count (e.g., '5 courses found'). Verify a screen reader announces updates when the count changes.

131. Add an aria-expanded attribute to any expandable element (e.g., a mobile hamburger menu). Toggle it between true and false on open/close.
132. Test keyboard navigation: Tab through all interactive elements in order. Ensure no interactive element is unreachable by keyboard. Fix any focus traps or skipped elements.

Hint:

- aria-live='polite' announces changes to screen readers without interrupting current speech. Use 'assertive' only for urgent messages.
- Never use outline: none on focused elements without providing an alternative visible focus indicator — keyboard users rely on it.

Expected Outcome: All interactive elements are reachable via Tab. Keyboard Enter on a course card works. Screen reader announces the results count update.

| Task 3: Colour Contrast & Cross-Browser Testing

Goal: Verify colour contrast ratios and test across different browsers.

Steps:

133. Use the Chrome DevTools colour picker or <https://webaim.org/resources/contrastchecker/> to check the contrast ratio of your primary text colour against its background. WCAG AA requires at least 4.5:1 for normal text.
134. Adjust any failing colour combinations to meet the 4.5:1 minimum. Document the before and after hex values in a comment.
135. Open the Student Portal in Firefox and Safari (or Edge). Note any visual differences (font rendering, flexbox gaps, grid behaviour).
136. Use <https://caniuse.com/> to check browser support for one CSS feature you used (e.g., CSS Grid, clamp(), gap in Flexbox). Document the support table findings in a comment.
137. Add a CSS polyfill for a feature with limited support: include the css-vars-ponyfill CDN script and verify it works in a simulated older browser using DevTools emulation.

Hint:

- WCAG AA contrast ratio: 4.5:1 for normal text, 3:1 for large text (18pt+ or 14pt+ bold).
- Feature detection (using @supports in CSS or typeof in JS) is safer than browser detection — test for the feature, not the browser.

Expected Outcome: All text/background combinations pass 4.5:1 contrast. A documented caniuse check is present in your comments. Layout is consistent across Chrome and Firefox.

HANDS-ON 10 [Advanced]**API Integration & Advanced State Management****Topics Covered:**

✓ Fetch API vs Axios (comparison)	✓ Centralised API Layer
✓ Redux Toolkit (advanced)	✓ NgRx (Angular)
✓ Pinia (Vue) — Advanced Patterns	✓ Error Boundaries & Global Error Handling

In this final hands-on, you will consolidate your API integration knowledge, build a proper centralised API service layer, and implement advanced state management patterns across all three frameworks — React (Redux Toolkit), Angular (NgRx concept), and Vue (Pinia).

NOTE	You may choose one framework (React, Angular, or Vue) for the implementation tasks in this exercise. Read through all three sections to understand the patterns across frameworks.
-------------	--

Task 1: Build a Centralised API Service Layer

Goal: Create a reusable, maintainable API layer that all components consume.

Steps:

138. Create an `api/` folder in your project. Inside it, create `apiClient.js` (or `apiClient.ts` for Angular) that configures a single Axios instance with a `baseUrl`, default headers, and timeout.
139. Create `courseApi.js` with exported functions: `getAllCourses()`, `getCourseById(id)`, `enrollStudent(studentId, courseId)`. Each function uses the configured Axios instance.
140. Add a response interceptor on the Axios instance that: (a) returns `response.data` directly (so callers receive data, not the Axios response wrapper), (b) catches errors and throws a standardised Error object with message and `statusCode`.
141. Add a request interceptor that attaches an Authorization header (use a hardcoded mock token for now).
142. Update all components to use the new API functions instead of inline `fetch/axios` calls. Verify nothing broke.

Hint:

- Centralising API calls makes it trivial to switch the base URL (dev vs prod), add auth tokens in one place, and standardise error handling.
- A good API layer means your components never see HTTP status codes — they only deal with data or error messages.

Expected Outcome: All API calls go through the centralised client. Changing the `baseUrl` in one file affects the entire app. Error messages are standardised.

Task 2: Advanced Redux Toolkit — Async Thunks (React)

Goal: Use `createAsyncThunk` to handle async API calls in Redux, keeping components clean.

Steps:

143. In your Redux slice, add an async thunk using `createAsyncThunk('courses/fetchAll', async () => { return await getAllCourses(); })`.
144. Handle the three thunk lifecycle actions in the slice's `extraReducers`: `pending` → set `loading: true`, `fulfilled` → populate courses array + `loading: false`, `rejected` → set error message + `loading: false`.
145. In `CoursesPage.jsx`, dispatch the thunk in `useEffect`: `dispatch(fetchAllCourses())`. Remove any direct API calls from the component.
146. Add selectors: create `selectCourses` and `selectCoursesLoading` selector functions. Use them with `useSelector` in components — components should not access store shape directly.
147. Test error handling: temporarily change the API URL to an invalid endpoint. Verify the rejected action fires and an error message appears in the UI.

Hint:

- `createAsyncThunk` eliminates boilerplate for loading/error state management — a very common pattern in real Redux apps.
- Selectors decouple components from store shape — if you reorganise the state, you update selectors in one place, not every component.

Expected Outcome: Courses load via the thunk. Loading and error states work correctly. Components use selectors, not direct state access.

| Task 3: NgRx Concept (Angular) & Pinia Advanced Patterns (Vue)

Goal: Understand NgRx architecture for Angular and advanced Pinia patterns for Vue.

Steps:

148. NgRx Concept (read and implement if using Angular): NgRx follows the same Redux pattern — Actions, Reducers, Selectors, and Effects. Create a `courses.actions.ts` with `loadCourses` action. Create a `courses.reducer.ts` that handles the action. Create a `CoursesEffect` that listens for `loadCourses` and calls `CourseService`. Document the data flow in a comment: Component → dispatch Action → Effect → API → Reducer → State → Selector → Component.
149. Pinia Advanced (implement if using Vue): In the enrollment store, add an async action `fetchAndEnroll(courseId)` that calls the API and then updates state in one action. Add a `$reset()` call to clear enrollment state. Use `storeToRefs(store)` in a component to destructure reactive state without losing reactivity.
150. For either framework: implement a global error handler — in React use an Error Boundary component wrapping the app; in Angular use an ErrorHandler service; in Vue use `app.config.errorHandler`. Log errors and show a fallback UI.
151. Write a brief comparison (in code comments or a README.md) of how each framework handles state management: React+Redux, Angular+NgRx, Vue+Pinia. Note the key differences in boilerplate, learning curve, and built-in tooling.

Hint:

- NgRx Effects handle side effects (API calls) outside reducers — reducers must be pure functions (no API calls, no side effects).
- `storeToRefs(store)` in Pinia extracts refs while keeping reactivity — plain destructuring breaks reactivity in Vue's Composition API.

Expected Outcome: Global error handler catches and displays errors gracefully. README or comments contain a meaningful framework comparison.